CNN Accelerator for Pothole Detection Using Verilog SoC Project

Venkata Durgesh	230907454
Gurudeep Sai	230907202
Sai Suneeth	230907176
Manudeep	230907436

Abstract

This project presents the design and implementation of a CNN accelerator for pothole detection using Verilog HDL. The accelerator processes a 64 × 64 input image and performs convolution, ReLU activation, max pooling, flattening, and final ANN-based classification. The design uses BRAM for image storage, line buffers for continuous pixel streaming, sliding windows for convolution, fixed-point arithmetic for efficient hardware implementation, and pipelining to improve throughput.

The CNN architecture consists of two convolutional layers. The first convolution layer uses 8 filters of size 3 × 3 and produces 62 × 62 × 8 feature maps. After ReLU activation and 2 × 2 max pooling, the output becomes 31 × 31 × 8. The second convolution layer uses 16 filters of size 3 × 3 × 8 and produces 29 × 29 × 16 feature maps. After ReLU, zero padding is applied to convert 29 × 29 into 30 × 30, followed by 2 × 2 max pooling to obtain 15 × 15 × 16 feature maps. These feature maps are flattened into a vector of 3600 values and passed to an ANN classifier with two output classes: pothole detected and pothole not detected.

The main focus of this project is the hardware implementation of CNN operations using line buffers, sliding windows, convolution units, ReLU blocks, pooling units, and pipelined dataflow. Instead of waiting for the complete output of one layer before starting the next layer, the design allows intermediate results to move to the next stage as soon as valid data is available. This makes the accelerator suitable for FPGA-based real-time image processing applications

1. Introduction

Convolutional Neural Networks are widely used in image classification, object detection, and computer vision applications. However, CNNs require a large number

of multiplication and addition operations, especially in convolution layers. When CNNs are implemented only in software, the processing time can be high, particularly for real-time applications.

To overcome this limitation, CNN operations can be accelerated using dedicated hardware. In this project, a CNN accelerator is designed using Verilog HDL. The accelerator is intended for pothole detection from a 64 × 64 image. The input image is stored in BRAM and then processed through convolution, activation, pooling, flattening, and ANN classification stages.

The project mainly focuses on implementing CNN operations in hardware using a streaming architecture. The image pixels are read from BRAM and sent into line buffers. These line buffers generate a 3 × 3 sliding window, which is multiplied with kernel values to perform convolution. The convolution output is passed through ReLU activation and then max pooling. The output of the first layer is directly streamed into the second layer without waiting for the entire first layer to finish.

This type of architecture is useful because it reduces unnecessary memory access and improves processing speed. The use of line buffers and sliding windows allows the accelerator to process image data continuously. The use of fixed-point arithmetic reduces hardware complexity compared to floating-point arithmetic.

2. Objective of the Project

The main objective of this project is to design a CNN accelerator using Verilog HDL for pothole detection.

The specific objectives are:

1. To read a 64 × 64 input image from BRAM.
2. To store incoming pixels into line buffers.
3. To generate a 3 × 3 sliding window for convolution.
4. To perform 3 × 3 convolution using fixed-point kernel values.
5. To implement ReLU activation in hardware.
6. To implement 2 × 2 max pooling in hardware.

7. To perform pipelined processing between CNN layers.
8. To process the output of the first layer directly into the second layer.
9. To flatten the final feature maps.
10. To classify the image using an ANN layer with two outputs: pothole detected and pothole not detected.

3. Proposed CNN Architecture

The proposed CNN architecture takes a 64 × 64 grayscale image as input. Since the input image has only one channel, the input size is 64 × 64 × 1.

The CNN contains two convolutional layers, two ReLU activation stages, two max pooling stages, one flattening stage, and one ANN classifier.

The architecture is shown below:

Input Image: 64 × 64 × 1

Conv Layer 1:

3 × 3 convolution with 8 filters

Output: 62 × 62 × 8

ReLU Layer 1: Output: 62 × 62 × 8

Max Pooling Layer 1: 2 × 2 max pooling with stride 2

Output: 31 × 31 × 8

Conv Layer 2:

3 × 3 convolution with 16 filters

Each filter has depth 8

Output: 29 × 29 × 16

ReLU Layer 2:

Output: 29 × 29 × 16

Padding: 29 × 29 × 16 is padded to 30 × 30 × 16

Max Pooling Layer 2: 2 × 2 max pooling with stride 2

Output: 15 × 15 × 16

Flatten: 15 × 15 × 16 = 3600 values

ANN Classifier:

3600 input values

2 output classes

The final two output classes are:

Class 0: Pothole not detected

Class 1: Pothole detected

4. Layer-Wise CNN Calculation

The first convolution layer takes a 64 × 64 image as input. The kernel size is 3 × 3, stride is 1, and no padding is used.

For a convolution operation, the output size is calculated as:

Output size = Input size - Kernel size + 1
For the first convolution layer:
Input size = 64 × 64

Kernel size = 3 × 3

Output size = 64 - 3 + 1= 62

Therefore, one kernel produces: 62 × 62

Since the first layer has 8 kernels, the output becomes: 62 × 62 × 8

After ReLU, the size remains the same: 62 × 62 × 8

After 2 × 2 max pooling with stride 2: 62 / 2 = 31

So the output after the first pooling layer is:31 × 31 × 8

The second convolution layer takes 31 × 31 × 8 as input. Since the input depth is 8, each filter in the second convolution layer has the size:3 × 3 × 8

The second layer has 16 such filters. Therefore, the output depth becomes 16.

For the second convolution layer:

Input size = 31 × 31

Kernel size = 3 × 3

Output size = 31 - 3 + 1=29

So the output after second convolution is: 29 × 29 × 16

After ReLU, the size remains: 29 × 29 × 16

Since 29 is an odd number, padding is applied before pooling. The 29 × 29 feature map is padded to:30 × 30 × 16

After 2 × 2 max pooling with stride 2: 30 / 2 = 15

So the output becomes: 15 × 15 × 16

After flattening: 15 × 15 × 16 = 3600

Therefore, the ANN receives 3600 input values and produces 2 output values.

5. Overall Hardware Architecture

The hardware architecture of the CNN accelerator is based on a streaming dataflow model. The input image is first stored in BRAM. The pixels are then read sequentially and loaded into line buffers. The line buffers generate the required rows for the sliding window. The sliding window is multiplied with kernel values to perform convolution.

The overall flow is:

64 × 64 Input Image

↓

BRAM

↓

Three Line Buffers

↓

3 × 3 Sliding Window

↓

Convolution Layer 1

↓

ReLU Activation

↓

Two Line Buffers for Pooling

↓

2 × 2 Max Pooling

↓

Three Line Buffers for Second Layer

↓

3 × 3 Sliding Window

↓

Convolution Layer 2

↓

ReLU Activation

↓

Two Line Buffers for Pooling

↓

2 × 2 Max Pooling

↓

Flattening

↓

ANN Classifier

↓

Pothole Detected / Not Detected

6. Input Image Storage Using BRAM

The input image size is 64 × 64. Therefore, the total number of pixels is: 64 × 64 = 4096 pixels

These 4096 pixels are stored in Block RAM. The BRAM is accessed using an address counter. On every clock cycle, one pixel is read from BRAM and given to the accelerator.

In the Verilog code, the BRAM is instantiated as:

```
blk_mem_gen_0 bram (
    .clka(clk),
    .ena(enable),
    .wea(1'b0),
    .addra(addr),
    .douta(pixel)
);
```

Here, addr is the address counter used to read the pixel values from memory. The signal enable controls whether the BRAM is active or not. Since this design reads the image from BRAM, the write enable signal wea is kept as zero.
The BRAM output is stored in the signal pixel. These pixel values are then sent into the line buffers.

The purpose of using BRAM is to store the full input image efficiently on FPGA. Instead of giving all pixels at once, the image is streamed one pixel at a time. This makes the design suitable for hardware implementation.

7. Line Buffer Design

For a 3 × 3 convolution, the accelerator needs three rows of image data at the same time. To achieve this, three line buffers are used.

In the Verilog code, the line buffers are declared as:

```
reg [17:0] linebuffer1 [63:0];

reg [17:0] linebuffer2 [63:0];

reg [17:0] linebuffer3 [63:0];
```

Each line buffer stores one row of the input image. Since the image width is 64 pixels, each line buffer has 64 locations.

The line buffers are used to hold three consecutive rows of the image. When a new pixel is read from BRAM, it is inserted into the first line buffer. The previous value of the first line buffer is shifted to the second line buffer, and the previous value of the second line buffer is shifted to the third line buffer.

The logic is shown in the code:

```
linebuffer1[count] <= pixel;
linebuffer2[count] <= linebuffer1[count];
linebuffer3[count] <= linebuffer2[count];
```

8. Sliding Window Generation

After the line buffers contain valid data, a 3 × 3 sliding window is generated. The sliding window holds the current 3 × 3 region of the image.

The window is declared as:

```
reg [17:0] window [2:0][2:0];
```

The rightmost column of the window receives new values from the three line buffers:

```
window[0][2] <= linebuffer3[count];
window[1][2] <= linebuffer2[count];
window[2][2] <= linebuffer1[count];
```

Then the existing window values are shifted left:

```
window[0][1] <= window[0][2];
window[0][0] <= window[0][1];
window[1][1] <= window[1][2];
window[1][0] <= window[1][1];
window[2][1] <= window[2][2];
window[2][0] <= window[2][1];
```

This creates a sliding effect. On every valid clock cycle, a new 3 × 3 window is formed. This window is then passed to the convolution unit.

The sliding window avoids the need to fetch nine pixels separately for every convolution operation. Instead, the window is continuously updated as the image is streamed.

9. Convolution Operation

The convolution operation is the main operation in a CNN. In this project, a 3 × 3 convolution is implemented in hardware.

For one 3 × 3 window and one 3 × 3 kernel, the convolution is calculated as:

Output = W00K00 + W01K01 + W02K02

+ W10K10 + W11K11 + W12K12

+ W20K20 + W21K21 + W22K22

This operation performs nine multiplications and eight additions to produce one convolution output.

The first convolution layer uses 8 different 3 × 3 kernels. Each kernel produces one 62 × 62 feature map. Therefore, the first convolution layer produces 8 feature maps.

The second convolution layer uses 16 filters. Since the input to the second layer has depth 8, each second-layer filter has size 3 × 3 × 8. This means each second-layer output value is produced by combining information from all 8 feature maps of the previous layer.

In the current Verilog implementation, selected kernel paths are implemented and verified. The same structure can be extended or reused to implement the full 8-kernel and 16-filter architecture.

10. Fixed-Point Arithmetic

CNNs are usually trained using floating-point numbers. However, floating-point arithmetic requires more hardware resources. To reduce hardware complexity, this design uses fixed-point arithmetic.

The kernel values are stored in fixed-point format. After multiplication, the result becomes larger because both pixel values and kernel values are multiplied. To bring the result back to the correct scale, the output is shifted right by 16 bits.

```
kernel1_1_out_reg <= kernel1_1_out >> 16;
```

The advantage of fixed-point arithmetic is that it uses fewer FPGA resources compared to floating-point arithmetic. It also improves processing speed and makes the design more suitable for hardware implementation.

11. ReLU Activation

After convolution, the output is passed through a ReLU activation function. ReLU stands for Rectified Linear Unit.

The ReLU function is: $ReLU(x) = max(0, x)$

In hardware, this is implemented by checking the sign bit of the convolution output.

12. Max Pooling Operation

After ReLU, max pooling is performed. Max pooling reduces the spatial size of the feature map while keeping the most important features.

This project uses 2×2 max pooling. For every 2×2 block, the maximum value is selected.

In the Verilog code, the 2×2 pooling window is declared as:

```
reg [31:0] poll_window1 [1:0][1:0];
```

13. Pipelining and Parallel Dataflow

One of the most important features of this project is pipelining.

In a normal software implementation, one layer may be fully completed before the next layer begins. For example, the complete first convolution output may be calculated first, then the full ReLU output, then the full pooling output, and only after that the second convolution layer starts.

In this hardware accelerator, the design works differently. It uses a streaming pipeline. As soon as the first valid convolution output is produced, it is passed to ReLU. After ReLU, the result is stored in pooling buffers. Once enough values are available for a 2×2 pooling window, max pooling starts. After the first pooling stage produces valid outputs, those outputs are loaded into the next set of line buffers for the second convolution layer.

Therefore, the second layer does not wait for the full first layer to finish. It starts working as soon as enough valid data is available from the first layer.

The pipelined flow is:

BRAM Read

↓

Line Buffer Filling

↓

Window Generation

↓

Convolution Layer 1

↓

ReLU 1

↓

Pooling 1

↓

Line Buffers for Layer 2

↓

Convolution Layer 2

↓

ReLU 2

↓

Pooling 2

All these stages can operate in parallel after the pipeline is filled.

This reduces the total latency of the CNN accelerator. It also improves hardware utilization because different parts of the circuit are active at the same time.

For example, while new pixels are still being read from BRAM, the first convolution layer can generate outputs. At the same time, earlier convolution outputs can be processed by ReLU and pooling. Similarly, after enough pooled values are available, the second convolution layer can begin.

This streaming and pipelined approach is one of the main advantages of hardware CNN accelerators.

14. First Layer Hardware Implementation

The first layer starts by reading pixels from BRAM. The pixels are inserted into three line buffers. After three rows and three columns are available, a valid 3×3 window is generated.

The first convolution layer uses 3 × 3 kernel values. The convolution output is generated by multiplying the window values with the kernel values and adding the products.

The output of the first convolution layer is valid only after the window contains proper data. The signal sum_valid is used to indicate when the convolution output is valid.

The first layer operations are:

1. Read pixel from BRAM
2. Load pixel into line buffers
3. Generate 3 × 3 window
4. Multiply window with kernel
5. Add multiplication results
6. Shift result by 16 bits for fixed-point scaling
7. Apply ReLU
8. Store result into pooling buffers
9. Perform 2 × 2 max pooling
10. Send pooled output to second layer buffers

In the complete CNN model, this process is done for 8 kernels, producing:

31 × 31 × 8

15. Second Layer Hardware Implementation

The second layer hardware implementation is similar to the first layer. Just like the first layer, the second layer also uses line buffers, a sliding window, convolution, ReLU activation, and max pooling. The main difference is that the input to the second layer comes from the output of the first pooling layer instead of directly from BRAM.

In the first layer, the input image is read from BRAM and loaded into three line buffers. In the second layer, the pooled output from the first layer is loaded into

another set of three line buffers. These line buffers are used to generate a new 3 × 3 sliding window for the second convolution operation.

The input to the second layer is: 31 × 31 × 8

This means the second layer receives 8 feature maps from the first layer. Therefore, each filter in the second convolution layer must have a depth of 8.

So each second-layer filter has the size: 3 × 3 × 8

The second convolution layer has 16 such filters. Therefore, the output depth becomes 16.

The second convolution output size is calculated as:

Input size = 31 × 31

Kernel size = 3 × 3

Stride = 1

Padding = 0

Output size = 31 - 3 + 1

Output size = 29

Therefore, the output after the second convolution layer is: 29 × 29 × 16

After convolution, ReLU activation is applied. Similar to the first layer, ReLU replaces all negative values with zero and keeps positive values unchanged.

After ReLU, the size remains: 29 × 29 × 16

Since 29 is an odd number, padding is applied before max pooling. The feature map is padded from: 29 × 29 × 16

to: 30 × 30 × 16

After padding, 2 × 2 max pooling with stride 2 is applied.

The pooling output size is: 30 / 2 = 15

Therefore, the final output after the second pooling layer is: 15 × 15 × 16

16. Padding in the Second Layer

Padding is required in the second layer because the output of the second convolution is 29 × 29 × 16. Since max pooling uses a 2 × 2 window with stride 2, the input size should be even for clean division.

If pooling is directly applied to 29 × 29, one row and one column will not form a complete 2 × 2 pooling window. To avoid this, zero padding is applied.

The feature map is padded as: 29 × 29 × 16 → 30 × 30 × 16

Here, zeros are added at the boundary of each feature map. Padding does not add new learned information, but it helps the pooling operation work properly.

After padding, 2 × 2 max pooling with stride 2 is applied: 30 × 30 × 16 → 15 × 15 × 16

Thus, padding ensures that the second pooling layer produces a proper 15 × 15 × 16 output.

17. Flattening Layer

After the second pooling layer, the output feature map size is: 15 × 15 × 16

This is a three-dimensional feature map. However, the ANN classifier requires a one-dimensional input vector. Therefore, the 3D feature map is converted into a 1D vector using flattening.

The number of values after flattening is: 15 × 15 × 16 = 3600

So, the flattened output contains 3600 values.

Flattening does not change the actual values. It only changes the arrangement of the data from a three-dimensional format into a one-dimensional format.

This flattened vector is then given as input to the ANN classifier.

18. ANN Classifier

The ANN classifier is the final stage of the CNN accelerator. It receives the flattened vector from the previous stage.

The input size of the ANN is: 3600

The ANN produces two output values:

1. Pothole detected
2. Pothole not detected

The ANN performs multiplication between the flattened input values and trained weights. Then it adds the bias values and produces two output scores.

The general ANN operation is: Output = Input × Weight + Bias

Since this project is used for binary classification, only two final output neurons are required. One output represents pothole detected, and the other output represents pothole not detected.

The class with the higher output value is selected as the final result.

19. Control Signals Used in the Design

Since this CNN accelerator uses pipelined hardware, control signals are required to manage valid data movement between different stages.

Some important control signals used in the design are:

Signal	**Purpose**
addr	Reads pixels from BRAM
enable	Enables or disables BRAM
count	Tracks column position during line buffer loading
delay	Handles initial BRAM and line buffer delay
done_writing	Indicates completion of input reading or stage processing
sum_valid	Indicates that convolution output is valid
poll_valid	Indicates valid data for pooling
buffer_count	Tracks positions in pooling buffers
first_poll	Indicates completion of first pooling stage
poll_count	Tracks pooled outputs from the first layer

poll_in_count	Tracks second-layer pooling input
second_poll_stop	Stops second pooling after required outputs are produced

These control signals are important because convolution and pooling outputs are not valid from the first clock cycle. A 3 × 3 convolution window becomes valid only after enough rows and columns are loaded. Similarly, a 2 × 2 pooling window becomes valid only after enough values are available.

20. Verilog Module Description

The main Verilog module of the project is named live.

The module has the following input:

```
module live(
   input clk
);
```

The clock signal controls all sequential operations in the design.

The main components inside the module are:

1. BRAM input memory
2. Kernel registers
3. First-layer line buffers
4. First-layer sliding window
5. First-layer convolution unit
6. ReLU logic
7. First-layer pooling buffers
8. First-layer max pooling unit
9. Second-layer line buffers
10. Second-layer sliding window
11. Second-layer convolution unit

12. Second-layer pooling buffers
13. Second-layer max pooling unit

The kernel values are stored in registers and initialized inside the Verilog code. The kernel values are represented using fixed-point format.

21. Current Implementation Scope

The complete CNN architecture requires 8 kernels in the first convolution layer and 16 filters in the second convolution layer.

However, in the current Verilog implementation, only selected kernel paths are implemented and verified. The design currently demonstrates the working of the main hardware blocks such as line buffers, sliding windows, convolution, ReLU, pooling, and pipelining.

The current implementation verifies:

1. Reading the 64 × 64 image from BRAM
2. Loading pixels into three line buffers
3. Generating a 3 × 3 sliding window
4. Performing fixed-point convolution
5. Applying ReLU activation
6. Performing 2 × 2 max pooling
7. Sending pooled output to the second layer
8. Performing second-layer convolution
9. Performing second-layer pooling
10. Demonstrating pipelined data movement

The current project uses one selected path to prove that the CNN accelerator architecture works correctly. The same design can later be extended to support all 8 first-layer kernels and all 16 second-layer filters.

There are two possible methods for extending the architecture.

The first method is parallel implementation. In this method, separate convolution hardware is used for every kernel and filter. This gives high speed but uses more FPGA resources.

The second method is time-multiplexed implementation. In this method, the same convolution hardware is reused for multiple kernels across different clock cycles. This reduces hardware usage but increases the number of clock cycles required.

22. Simulation and Verification

The design is verified using simulation. The main purpose of simulation is to check whether each hardware block is producing the correct output at the correct time.

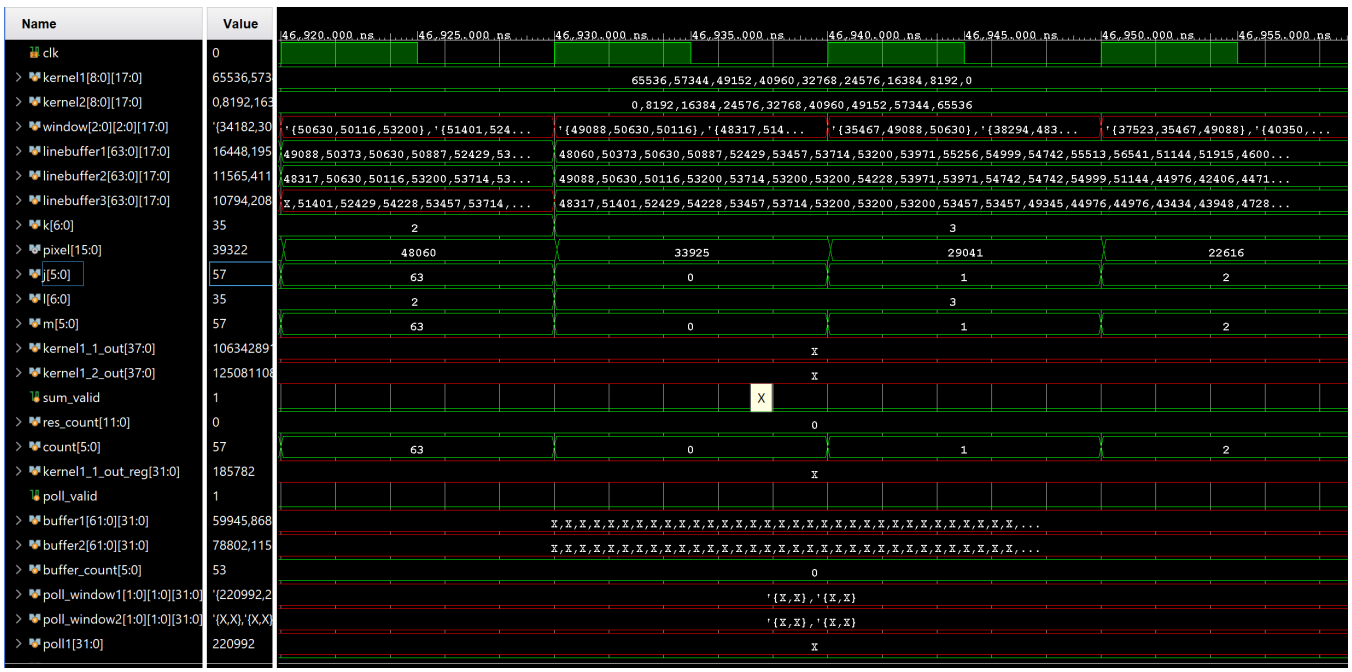

Fig1-input pixels loading from bram to linebuffers

The following signals are observed during simulation:

1. BRAM address counter
2. Pixel output from BRAM
3. Line buffer values
4. Sliding window values
5. First convolution output

6. ReLU output
7. First pooling output
8. Second-layer line buffer values
9. Second-layer sliding window values
10. Second convolution output
11. Second pooling output

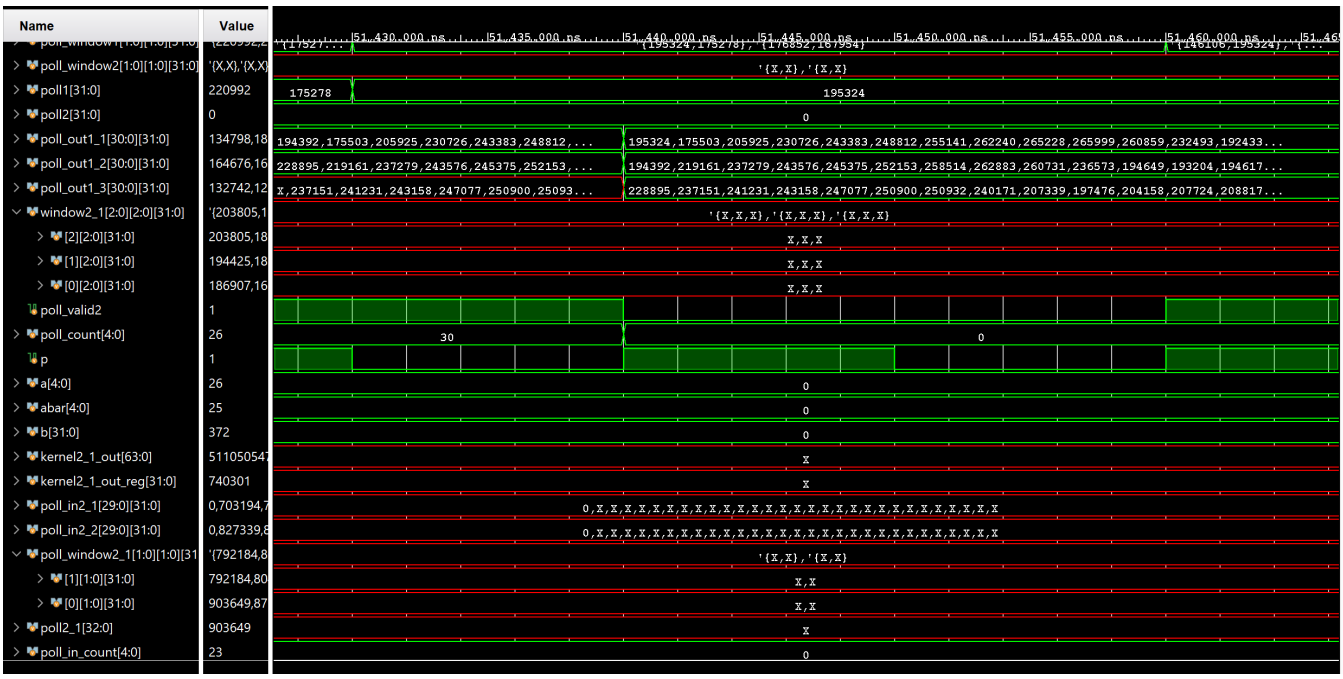

Fig2- after 1st layer polling the values enter into another set of line buffers for 2nd layer convolution

The expected output of the first convolution layer for one kernel is: 62 × 62 = 3844 values

The expected output after first pooling for one feature map is: 31 × 31 = 961 values

The expected output after second convolution for one feature map is: 29 × 29 = 841 values

After padding and second pooling, the expected output for one feature map is: 15 × 15 = 225 values

For 16 feature maps, the final output before flattening is: 15 × 15 × 16 = 3600 values

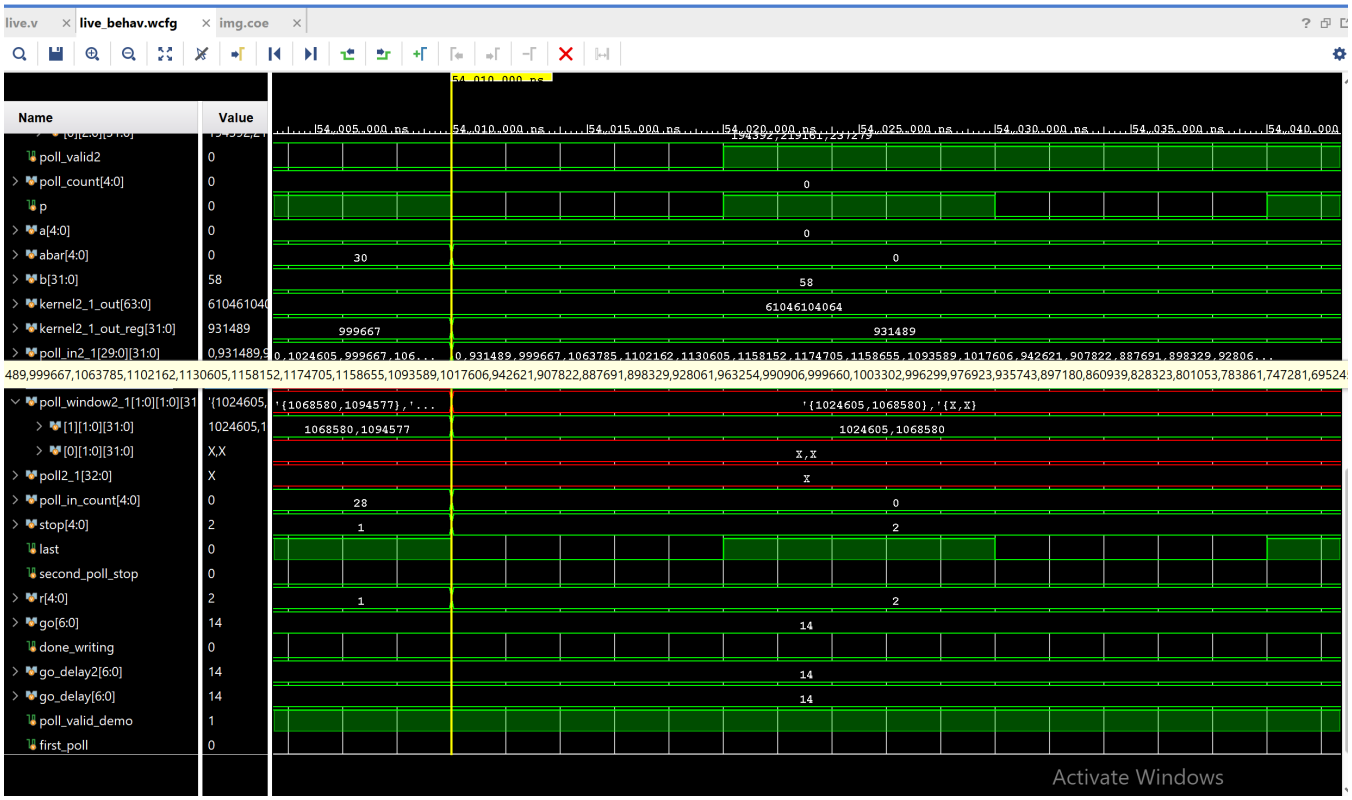

Fig3 – after 2nd layer convolution the values are entering into 2 line buffers for polling

Manual calculations are also performed for selected windows. The convolution output and pooling output from simulation are compared with manually calculated values to verify correctness.

23. Advantages of the Proposed Design

The proposed CNN accelerator has several advantages.

The first advantage is that it uses a streaming architecture. Pixels are read from BRAM and processed continuously instead of storing and processing the full image repeatedly.

The second advantage is the use of line buffers. Line buffers allow the accelerator to generate a 3 × 3 window efficiently without reading nine pixels separately from memory every time.

The third advantage is the use of fixed-point arithmetic. Fixed-point arithmetic requires fewer hardware resources than floating-point arithmetic and is more suitable for FPGA implementation.

The fourth advantage is pipelining. Different stages of the CNN can work at the same time. While one stage is reading pixels, another stage can perform convolution, another stage can perform pooling, and another stage can prepare data for the next layer.

The fifth advantage is modularity. The design can be divided into separate modules such as BRAM reading, line buffer, window generation, convolution, ReLU, pooling, flattening, and ANN. This makes the design easier to expand and modify.

24. Limitations of the Current Design

The current design has some limitations.

The first limitation is that the full CNN architecture is not completely implemented yet. The complete architecture requires 8 kernels in the first layer and 16 filters in the second layer, but the current implementation verifies selected kernel paths.

The second limitation is that the kernel values are hardcoded inside the Verilog code. A more flexible design should load trained weights from memory.The third limitation is that the ANN classifier is not fully integrated in the current implementation. The current focus is mainly on convolution, ReLU, pooling, and pipelining.

The fourth limitation is that fixed-point arithmetic may cause small accuracy loss compared to floating-point arithmetic.

The fifth limitation is that the control logic becomes complex because valid signals are required for each stage. Boundary handling, padding, and pooling control must be carefully designed.

25. Future Scope

The project can be improved by completing the full CNN architecture with all 8 first-layer kernels and all 16 second-layer filters. The ANN classifier can also be fully implemented in Verilog. In future work, the trained weights can be stored in BRAM or external memory instead of being hardcoded. The design can also be synthesized on FPGA to analyze resource usage such as LUTs, flip-flops, DSP blocks, and BRAM. Finally, the hardware output can be compared with a software CNN model, and the design can be optimized for larger images, deeper CNN layers, and better performance.

26. Results

The CNN accelerator successfully demonstrates the main operations required for hardware-based CNN processing. The input image is read from BRAM and streamed into line buffers. The line buffers generate a 3 × 3 sliding window, which is used for convolution.

The convolution output is passed through ReLU activation. After ReLU, the values are sent to pooling buffers, where 2 × 2 max pooling is performed.

The first layer reduces the image from 64 × 64 to 31 × 31 feature maps after convolution and pooling. The output of the first pooling layer is then directly sent into the second-layer line buffers.

This shows the pipelined nature of the design. The second layer does not wait for the full first layer to complete. Instead, it starts processing as soon as enough valid pooled values are available.

The second layer performs convolution, ReLU, padding, and pooling. The final output of the complete CNN architecture is expected to be 15 × 15 × 16. After flattening, this becomes 3600 values for the ANN classifier.

The main result of the project is that the basic CNN accelerator dataflow is implemented using Verilog. The design proves that convolution, ReLU, pooling, and pipelining can be implemented using hardware structures such as BRAM, line buffers, sliding windows, counters, and valid signals.

27. Conclusion

This project presents the design of a Verilog-based CNN accelerator for pothole detection. The accelerator processes a 64 × 64 input image using convolution, ReLU activation, max pooling, flattening, and ANN classification.

The second layer is implemented in a way similar to the first layer, using line buffers, sliding windows, convolution, ReLU, and pooling. The main difference is that the second layer receives feature maps from the first pooling layer instead of raw image pixels from BRAM. The complete CNN architecture produces 15 × 15 × 16 feature maps after the second pooling layer. These are flattened into 3600 values and passed to an ANN classifier with two outputs: pothole detected and pothole not detected.

The current implementation verifies selected kernel paths and demonstrates the main CNN accelerator flow. The design can be extended to implement all 8 kernels in the first layer and all 16 filters in the second layer.

The use of BRAM, line buffers, sliding windows, fixed-point arithmetic, ReLU, max pooling, and pipelining makes the design suitable for FPGA-based image processing applications.